AI Usage

Chenjie Xu

AI Use Report

This document outlines how AI tools (ChatGPT) were used throughout the development of the MistBar Tarot Cocktail Experience project. AI was primarily used as a support tool for problem-solving, logic structuring, debugging, and refining both code and written content.

Overview of AI Usage

AI was used in the following areas:

- Structuring website development workflow
- Debugging JavaScript logic and fixing interaction issues
- Generating reusable code patterns (navigation, progress bar, event handling)
- Integrating external APIs and handling data
- Designing filtering logic for user selections
- Supporting written content (UX report, proposal, design documentation)
- Assisting with UI/UX reasoning and design decisions

All final implementation decisions and adaptations were made by the author.

Representative Prompts and Usage
<u>1. Structuring Website Development Workflow</u>

Prompt:
“What’s the most logical flow of writing code to build a website using HTML, CSS, and JavaScript?”

Usage:
This helped establish a clear development order:

- HTML structure first
- CSS styling second
- JavaScript logic last

This structure was followed throughout the project, improving organization and maintainability.

<u>2. Navigation and Layout Design</u>

Prompt:
“Show me a simple and clean CSS template for a navigation bar.”

Usage:
AI provided a base structure for a minimal navigation system.
Follow-up explanations clarified:

- The use of `position: sticky` to maintain persistent navigation
- The use of `z-index` to ensure proper layering

These were later implemented to improve usability and interface stability.

```
CSS

.navbar {
  display: flex;
  justify-content: space-between;
  align-items: center;
  padding: 16px 32px;
  background-color: #ffffff;
  border-bottom: 1px solid #e5e5e5;
  position: sticky;
  top: 0;
  z-index: 100;
}
```

3. Progress Bar Implementation

Prompt:
“Show me how to make a progress bar for a quiz.”

Usage:
AI provided a basic HTML/CSS structure that was adapted into the quiz flow.
This helped visually communicate progress to users and improved engagement.

```html
<div class="progress-bar">
  <div class="progress-fill" style="width: 33.33%;"></div>
</div>
```

```css
.progress-bar {
  width: 100%;
  height: 8px;
  background: #f1ecf8;
  border-radius: 10px;
  overflow: hidden;
}

.progress-fill {
  height: 100%;
  background: linear-gradient(90deg, #7d6ca6, #9b8ac7);
  transition: width 0.5s ease;
  border-radius: 10px;
}
```

4. Screen Navigation Logic

Prompt:
“How can I hide screens and show one using JavaScript without multiple HTML files?”

Usage:
This led to the implementation of a reusable `showScreen()` function to control page states (quiz, ingredient flow, results).
The final version was simplified and adapted for better readability and control.

```javascript
function showScreen(id) {
  const screens = document.querySelectorAll(".screen");

  screens.forEach(function(screen) {
    screen.classList.remove("active");
  });

  const target = document.getElementById(id);
  target.classList.add("active");
}
```

5. Event Handling for Quiz Flow

Prompt:
“How do I use event listeners to move between quiz questions?”

Usage:
AI provided a structure for attaching event listeners to user actions.
This was adapted to handle user selections and transitions between quiz steps.

```javascript
const question1Buttons = document.querySelectorAll('[data-question="1"]');

question1Buttons.forEach(function(button) {
  button.addEventListener("click", function() {
    const answer1 = button.getAttribute("data-value");
    showScreen("question2");
  });
});
```

6. Debugging and Problem Solving

A major issue occurred where selecting new ingredients resulted in the same cocktail output.

Problem:
The result did not update when new selections were made.

AI-assisted Solution:

- Identified mismatch between UI state and internal data state
- Suggested re-rendering UI components
- Helped implement state reset logic

The final fix involved:

- Re-rendering ingredient cards
- Clearing outdated selections
- Adding debugging logs

This significantly improved system accuracy and reliability.

```javascript
async function getRandomCocktail() {
  const response = await fetch("https://www.thecocktaildb.com/api/json/v1/1/random.php");
  const data = await response.json();

  const drink = data.drinks[0];

  document.querySelector("#drink-name").textContent = drink.strDrink;
  document.querySelector("#drink-image").src = drink.strDrinkThumb;
  document.querySelector("#drink-instructions").textContent = drink.strInstructions;
}
```

7. Dynamic Content Generation

Prompt:
“Help me structure data into an array and generate HTML dynamically.”

Usage:
AI helped organize large datasets and suggested a pattern for dynamically generating UI components using JavaScript.

This allowed the project to:

- Avoid repetitive HTML
- Build scalable content systems
- Render results dynamically

```javascript
‹/› JavaScript

async function getCocktailsByIngredient(ingredient) {
  const url = `https://www.thecocktaildb.com/api/json/v1/1/filter.php?i=${ingredient}`;
  const response = await fetch(url);
  const data = await response.json();

  return data.drinks;
}
```

8. Writing and Documentation Support

AI was used to assist in writing:

- UX Report
- Project Proposal
- Design Documentation
- Citations

It helped refine structure, clarity, and tone, while all content decisions remained author-driven.

```javascript
‹/› JavaScript

const selectedIngredients = new Set();

function toggleIngredient(ingredient) {
  if (selectedIngredients.has(ingredient)) {
    selectedIngredients.delete(ingredient);
  } else {
    selectedIngredients.add(ingredient);
  }

  console.log(Array.from(selectedIngredients));
}
```

9. API Integration (Cocktail Data Retrieval)

Prompt:
“How can I use an API to fetch cocktail data and display it dynamically on my website?”

Usage:
AI was used to understand how to integrate external APIs into the project. It provided guidance on:

- Using `fetch()` to retrieve data
- Handling JSON responses
- Accessing and displaying relevant fields (e.g., drink name, images, ingredients)

This supported the integration of **TheCocktailDB API**, enabling dynamic data retrieval instead of static content.

The final implementation included:

- Fetching cocktail data from API endpoints
- Storing and processing responses
- Dynamically rendering results in the interface

```javascript
function renderIngredientCards(ingredients) {
  const container = document.querySelector("#ingredient-list");
  container.innerHTML = "";

  ingredients.forEach(function(ingredient) {
    const card = document.createElement("button");
    card.className = "ingredient-card";
    card.textContent = ingredient;

    card.addEventListener("click", function() {
      toggleIngredient(ingredient);
      card.classList.toggle("active");
    });

    container.appendChild(card);
  });
}
```

10. Cocktail Filtering Logic (Ingredient-Based Selection)

Prompt:
“How can I filter cocktails based on selected ingredients and update results dynamically?”

Usage:
AI helped structure the logic for ingredient-based filtering. It provided guidance on:

- Storing user selections (e.g., using Sets or arrays)
- Comparing selected ingredients with available data
- Filtering and updating results dynamically

This was implemented in the **By Ingredients** feature.

During development, AI also helped debug a key issue where:

- Data state was updated
- But UI state was not synchronized

The solution involved:

- Re-rendering UI components
- Synchronizing selection state
- Improving filtering accuracy

This enhanced both usability and system reliability.

```javascript
window.debugIngredientState = function() {
  console.log("Selected ingredients:", Array.from(selectedIngredients));
  console.log("Number selected:", selectedIngredients.size);
  console.log("Current results:", mixedDrinkCards);
  console.log("Current result index:", currentMixedDrinkIndex);
};
```

Reflection on AI Use

AI functioned as a support tool rather than a replacement for decision-making. It was most valuable for:

- Explaining technical concepts
- Providing structured starting points
- Debugging complex issues

However, all outputs required interpretation, modification, and integration. The final system design, logic, and user experience decisions were independently developed.

Conclusion

The use of AI in this project improved efficiency, supported problem-solving, and enhanced both technical and design processes. By combining AI assistance with independent decision-making, the project successfully balances functionality, interaction design, and user experience.